NESNE YÖNELİMLİ PROGRAMLAMA - 1

Sınıf ve Örnek Değişkenleri

Paylaşılan veri, kişiye özel veri ve metot türleri

`self.x` `Sinif.x` `@classmethod` `@staticmethod`

```
ogrenci.py
class Ogrenci:
    sayac = 0        # herkes erişir

    def __init__(self, ad):
        self.ad = ad  # kişiye özel
        Ogrenci.sayac += 1
```

Erhan AKAGÜNDÜZ

BU HAFTA

Dersin sonunda neler yapabileceksiniz?

Örnek değişkeni

Her nesneye özel veriyi self üzerinde tutmak

Sınıf değişkeni

Tüm nesnelerin paylaştığı veriyi tanımlamak ve doğru güncellemek

Arama sırası

Python'un bir niteliği nerede aradığını ve gölgeleme tuzağını açıklamak

Metot türleri

Örnek, sınıf ve statik metot arasında doğru seçim yapmak

Renk kodu: **yeşil** = nesneye özel, **turuncu** = sınıfta ortak, **mor** = metotlar.

GEÇEN HAFTADAN BUGÜNE

Her nesne kendi verisini taşıyor. Peki ortak veri?

ada

```
ad = "Ada"
okul = "Python Akademi"
```

erhan

```
ad = "Erhan"
okul = "Python Akademi"
```

nil

```
ad = "Nil"
okul = "Python Akademi"
```

? Tekrar eden veri

Okul adı herkes için aynı. 300 öğrencide 300 kopya mı? Okul adı değişirse 300 nesneyi tek tek mi güncelleyeceğiz?

? Kimseye ait olmayan veri

Toplam öğrenci sayısı Ada'nın mı, Erhan'nın mı niteliği olmalı? Hiçbir nesne tek başına bilemez.

BÖLÜM 1

01 Örnek değişkenleri

Instance variables: her nesnenin kendine ait verisi.

self.x | nesne başına bir kopya | bellek görünümü

self üzerine yazılan her şey nesneye özeldir

ornek.py

```python
class Ogrenci:
    def __init__(self, ad, bolum):
        self.ad = ad          # örnek değişkeni
        self.bolum = bolum    # örnek değişkeni

a = Ogrenci("Ada", "Yazılım")
b = Ogrenci("Erhan", "Elektrik")
b.bolum = "Makine"
print(a.ad, a.bolum)
print(b.ad, b.bolum)
```

▷ ÇIKTI

```
Ada Yazılım
Erhan Makine
```

Örnek değişkeni (instance variable)

Genellikle **__init__** içinde **self.isim = değer** ile tanımlanır. Her nesnede **ayrı bir kopyası** vardır.

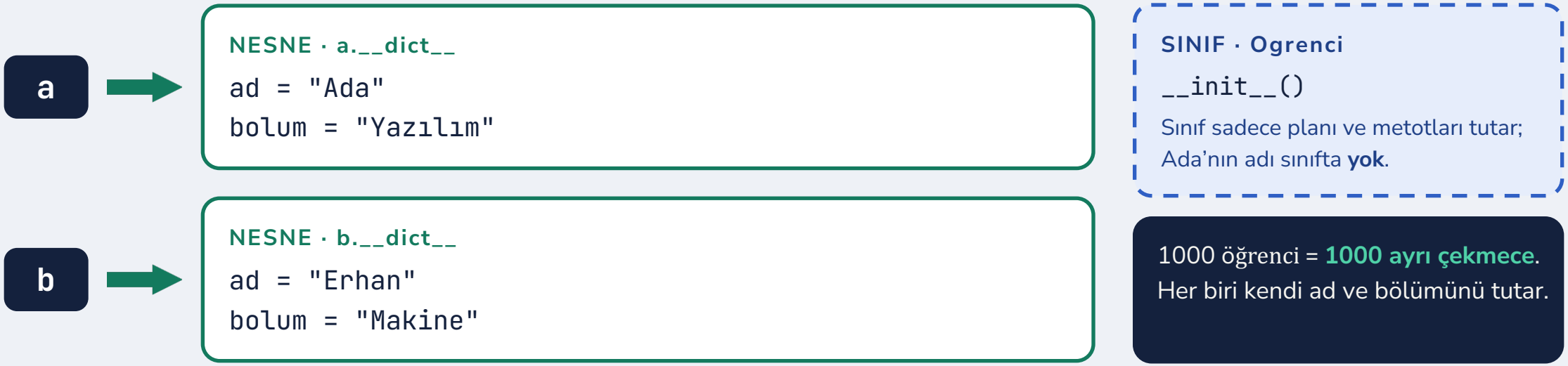
b.bolum değişti, **a.bolum** etkilenmedi.

Erişim: **nesne.isim** (dışarıdan) veya **self.isim** (metot içinden)

Ömrü: nesne yaşadığı sürece

ÖRNEK DEĞİŞKENİ · BELLEKTE

Her nesnenin kendi çekmecesi var



Birazdan sınıf kutusunun içine de veri koyacağız: o veri **herkesin ortak çekmecesi** olacak.

BÖLÜM 2

02

Sınıf değişkenleri

Class variables: tüm nesnelerin paylaştığı tek bir kopya

ortak veri · sayaç · arama sırası · gölgeleme · üç tuzak

Global değişkenle saymak: çalışır ama yanlış yerde

global_sayac.py

```
toplam = 0
class Ogrenci:
    def __init__(self, ad):
        global toplam
        self.ad = ad
        toplam += 1

Ogrenci("Ada"); Ogrenci("Efe"); Ogrenci("Nil")
print("Toplam:", toplam)
```

▷ ÇIKTI
```
Toplam: 3
```

Neden iyi değil?

- ⚠ Öğrencilerle ilgili bilgi **sınıfın dışında** duruyor.
- ⚠ Herhangi bir kod **toplam = 99** yazıp bozabilir.
- ⚠ Başka bir sınıf da **toplam** adını kullanırsa çakışır.

1. haftadan hatırlayın: **global** son çare. Ortak veri **sınıfın içinde** durmalı.

Sınıfın gövdesinde, metotların dışında

sinif_degiskeni.py

```python
class Ogrenci:
    okul = "Python Akademi"    # sınıf değişkeni
    def __init__(self, ad):
        self.ad = ad           # örnek değişkeni
a = Ogrenci("Ada")
b = Ogrenci("Erhan")
print(Ogrenci.okul)
print(a.okul, "|", b.okul)
Ogrenci.okul = "Kod Okulu"
print(a.okul, "|", b.okul)
```

▷ ÇIKTI

```
Python Akademi
Python Akademi | Python Akademi
Kod Okulu | Kod Okulu
```

Sınıf değişkeni (class variable)

Sınıf gövdesinde, hiçbir metodun içinde olmadan tanımlanır. Bellekte **tek bir kopyası** vardır ve tüm nesneler onu görür.

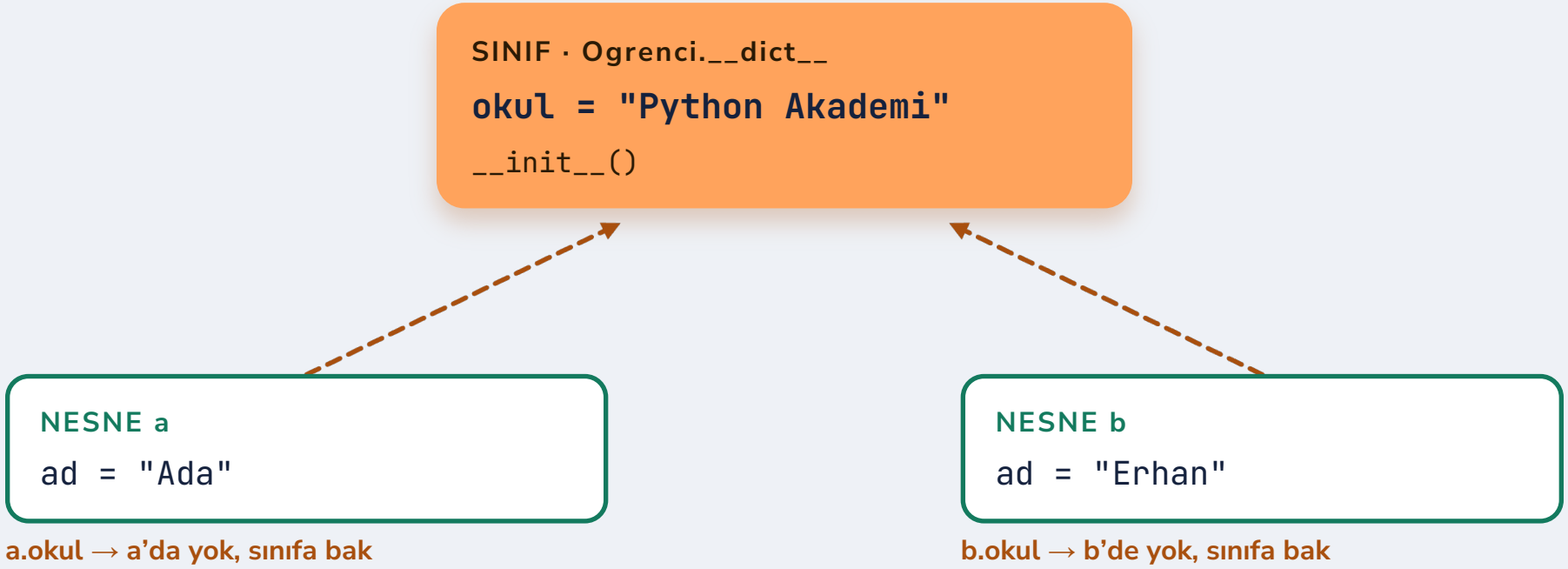
Sınıf üzerinden değiştirince **herkes** yeni değeri görür.

Okumak için: **Ogrenci.okul** veya **a.okul** (ikisi de çalışır)

Değiştirmek için: **her zaman Ogrenci.okul = ...**

SINIF DEĞİŞKENİ · BELLEKTE

Bir ortak çekmece, birçok kişisel çekmece



"okul" nesnelerin içinde **kopyalanmaz**. a.okul yazınca Python önce a'ya bakar, bulamazsa **sınıfa** çıkar.

SINIF DEĞİŞKENİ · SAYAÇ

Sayaç, olması gereken yerde: sınıfın içinde

sayac.py

```
class Ogrenci:
    sayac = 0

    def __init__(self, ad):
        self.ad = ad
        Ogrenci.sayac += 1
        self.no = Ogrenci.sayac

a = Ogrenci("Mustafa Şiyar")
b = Ogrenci("Ömer")
c = Ogrenci("Yamaç")
print("Toplam:", Ogrenci.sayac)
print(a.no, b.no, c.no)
```

▷ ÇIKTI

```
Toplam: 3
1 2 3
```

Satır	Ogrenci.sayac	self.no
sınıf tanımı	0	–
Ogrenci("Mustafa Şiyar")	1	a.no = 1
Ogrenci("Ömer")	2	b.no = 2
Ogrenci("Yamaç")	3	c.no = 3

İki tür birlikte çalışıyor: **sayac** herkesin, **no** her öğrencinin kendi numarası.

ARAMA SIRASI

a.okul yazınca Python nereye bakar?

arama.py

```
class Ogrenci:
    okul = "Python Akademi"
    def __init__(self, ad):
        self.ad = ad

a = Ogrenci("Ada")
print(a.__dict__)
print("okul" in a.__dict__)
print("okul" in Ogrenci.__dict__)
print(a.okul)
```

▷ ÇIKTI

```
{'ad': 'Ada'}
False
True
Python Akademi
```

1. **Nesnenin** kendi sözlüğüne bak. Varsa al, bitti.

2. Yoksa **sınıfın** sözlüğüne bak.

3. Orada da yoksa **AttributeError**.

1. haftadaki LEGB gibi: önce yakına, sonra uzağa. (9. haftada kalıtımla bir basamak daha eklenecek.)

TUZAK 1 · GÖLGELEME

Nesne üzerinden atama, sınıfı değiştirmez

golge.py

```
class Ogrenci:
    okul = "Python Akademi"
    def __init__(self, ad):
        self.ad = ad
a = Ogrenci("Ada")
b = Ogrenci("Efe")
a.okul = "Başka Okul"    # dikkat!
print(a.okul)
print(b.okul)
print(Ogrenci.okul)
print(a.__dict__)
```

▷ ÇIKTI

```
Başka Okul
Python Akademi
Python Akademi
{'ad': 'Ada', 'okul': 'Başka Okul'}
```

a

```
ad = "Ada"
okul = "Başka…"
```

b

```
ad = "Efe"
```

a.okul = … sınıfı değiştirmedi; a'ya **yeni bir örnek değişkeni** ekledi. Artık a, sınıftakini **gölgeliyor**.

Kural: okurken yukarı bakılır, **yazarken asla**.

TUZAK 2 · self.sayac += 1

Sayaç neden hep sıfır kaldı?

self_sayac.py

```
class Ogrenci:
    sayac = 0
    def __init__(self, ad):
        self.ad = ad
        self.sayac += 1   # HATA: self ile
a = Ogrenci("Ada")
b = Ogrenci("Efe")
print(Ogrenci.sayac)
print(a.sayac, b.sayac)
```

▷ ÇIKTI

```
0
1 1
```

AÇILIMI

`self.sayac = self.sayac + 1`

1. **Sağ taraf okur:** nesnede yok, sınıftan **0** gelir.
2. **Sol taraf yazar:** sonucu (1) **nesneye** yazar. Sınıf hiç değişmez.

`Ogrenci.sayac += 1` ← doğrusu

Ada’nın notu neden Efe’de görünüyor?

liste_tuzak.py

```
class Ogrenci:
    notlar = []              # sınıf değişkeni!
    def __init__(self, ad):
        self.ad = ad
    def not_ekle(self, n):
        self.notlar.append(n)

a = Ogrenci("Ada")
b = Ogrenci("Efe")
a.not_ekle(90)
print(b.notlar)
print(a.notlar is b.notlar)
```

▷ ÇIKTI

```
[90]
True
```

Neden gölgeleme olmadı?

append bir atama değil. self.notlar okunur (sınıftaki liste bulunur) ve o **tek liste yerinde değiştirilir**. Herkes aynı listeyi görür.

Düzeltme

```
def __init__(self, ad):
    self.ad = ad
    self.notlar = []
```

KARŞILAŞTIRMA

Örnek değişkeni ve sınıf değişkeni yan yana

Özellik	Örnek değişkeni	Sınıf değişkeni
Nerede tanımlanır?	__init__ içinde: self.x = ...	Sınıf gövdesinde: x = ...
Kaç kopya?	Nesne başına bir	Sınıf başına bir
Kime ait?	Yalnızca o nesneye	Tüm nesnelere ortak
Nerede durur?	nesne.__dict__	Sinif.__dict__
Nasıl değiştirilir?	self.x = ... / nesne.x = ...	Sinif.x = ...
Tipik örnekler	ad, bakiye, notlar	okul, sayac, kdv_orani

🔑 Soru şu: **“Bu bilgi her nesnede farklı olabilir mi?”** Evet → örnek değişkeni. Hayır, herkes için aynı → sınıf değişkeni.

KARAR VER

Bu bilgi nerede durmalı?

KDV oranı

Tüm ürünlerde aynı

Sınıf değişkeni

Ürünün fiyatı

Her üründe farklı

Örnek değişkeni

Üretilen ürün sayısı

Hiçbir ürüne ait değil

Sınıf değişkeni

Hesap bakiyesi

Her hesapta farklı

Örnek değişkeni

Asgari ücret

Tüm çalışanlar için tek değer

Sınıf değişkeni

Öğrencinin notları

Kişiye özel liste

Örnek değişkeni

Sınıf değişkenleri genellikle **sabitler**, **sayaçlar** ve **ortak ayarlar** içindir.

BÖLÜM 3

03

Metot tanımlama

Üç tür metot: nesneyle, sınıfla ya da hiçbiriyle çalışanlar

örnek metodu · self | @classmethod · cls | @staticmethod | hangisini seçmeli?

İlk parametreye bak, türü anla

Örnek metodu

```
def bilgi(self):
    ...
```

İlk parametre: self (nesne)

Erişir: nesnenin ve sınıfın verisi

Çağrı: a.bilgi()

En yaygın tür

Sınıf metodu

```
@classmethod
def toplam(cls):
```

İlk parametre: cls (sınıf)

Erişir: yalnızca sınıfın verisi

Çağrı: Ogrenci.toplam()

Sayaçlar, alternatif yapıcılar

Statik metot

```
@staticmethod
def gecerli(n):
```

İlk parametre: özel yok

Erişir: sadece kendi parametreleri

Çağrı: Ogrenci.gecerli(5)

Sınıfla ilgili yardımcılar

Dekoratör: metodun üstüne yapıştırılan etiket

```
# kısa yazım
@classmethod
def toplam(cls):
    return cls.sayac
```

=

```
# uzun yazım (aynı anlam)
def toplam(cls):
    return cls.sayac
toplam = classmethod(toplam)
```

Ne işe yarar?

Dekoratör, altındaki fonksiyonun **nasıl davranacağını değiştirir**. Fonksiyonun içine dokunmadan ona yeni bir özellik ekler.

@classmethod → "ilk parametreye nesneyi değil, **sınıfı** gönder"

@staticmethod → "ilk parametreye **hiçbir şey** gönderme"

Dekoratör yok → varsayılan: "ilk parametreye **nesneyi** gönder" (self)

5. haftada **@property** dekoratörünü de göreceğiz.

Bir metot ya değiştirir ya da bilgi verir

sorgu_komut.py

```python
class Hesap:
    def __init__(self, bakiye):
        self.bakiye = bakiye
    def yatir(self, miktar):          # komut
        self.bakiye += miktar
    def yeterli_mi(self, miktar):     # sorgu
        return self.bakiye >= miktar

h = Hesap(100)
print(h.yatir(50))
print(h.bakiye)
print(h.yeterli_mi(200))
```

▷ ÇIKTI

```
None
150
False
```

Komut

Durumu değiştirir, genelde **None** döner.

Sorgu

Durumu değiştirmez, bir **değer döndürür**.

⚠ **print(h.yatir(50))** → None. Komutun sonucunu yazdırmaya çalışmak sık bir hata.

cls: metodun içinden “bu sınıf” demek

classmethod.py

```python
class Ogrenci:
    sayac = 0
    def __init__(self, ad):
        self.ad = ad
        Ogrenci.sayac += 1

    @classmethod
    def toplam(cls):
        return f"{cls.__name__}: {cls.sayac} kayıt"

Ogrenci("Ada"); Ogrenci("Efe")
print(Ogrenci.toplam())
```

▷ ÇIKTI `Ogrenci: 2 kayıt`

self gibi, ama sınıf için

Ogrenci.toplam() çağrılınca Python, **cls** parametresine nesneyi değil **Ogrenci sınıfının kendisini** gönderir.

`Ogrenci.toplam()`

perde arkasında ≈

`toplam(Ogrenci)`

Sınıf metodu **self**'e erişemez; çünkü belirli bir öğrenci için değil, **tüm sınıf** için çalışır.

Nesne üretmenin ikinci kapısı

fabrika.py

```python
class Ogrenci:
    def __init__(self, ad, numara):
        self.ad = ad
        self.numara = numara

    @classmethod
    def metinden(cls, metin):
        ad, numara = metin.split(",")
        return cls(ad, int(numara))

a = Ogrenci.metinden("Ada,101")
print(a.ad, a.numara, type(a.numara))
```

▷ ÇIKTI `Ada 101 <class 'int'>`

Neden kullanılır?

Veri farklı biçimlerde gelebilir: bir dosya satırı, bir sözlük, bir CSV. Her biçim için ayrı bir "fabrika metodu" yazarız; hepsi sonunda **cls(...)** ile normal __init__'i çağırır.

`"Ada,101"` → `metinden()`

`{"ad": "Ada", …}` → `sozlukten()`

Python'da da var: **dict.fromkeys()** bir sınıf metodudur.

STATİK METOT · @staticmethod

Ne self ne cls: sınıfın içindeki yardımcı

staticmethod.py

```python
class Ogrenci:
    def __init__(self, ad, numara):
        if not Ogrenci.gecerli_numara(numara):
            raise ValueError("Geçersiz numara")
        self.ad = ad
        self.numara = numara

    @staticmethod
    def gecerli_numara(n):
        return 100 <= n <= 999

print(Ogrenci.gecerli_numara(101))
print(Ogrenci.gecerli_numara(42))
a = Ogrenci("Ada", 101)
print(a.ad)
```

▷ ÇIKTI

```
True
False
Ada
```

Neden sınıfın içinde?

gecerli_numara ne nesneye ne sınıf değişkenine ihtiyaç duyar. Normal bir fonksiyon da olabilirdi; ama **konusu öğrenci** olduğu için sınıfın içinde durması kodu düzenli tutar.

✓ Nesne oluşturmadan kullanılabilir: form doğrulama, birim dönüştürme, hesaplama yardımcıları.

Kim nereden çağrılabilir?

cagirma.py

```python
class Ogrenci:
    sayac = 0
    def __init__(self, ad):
        self.ad = ad
        Ogrenci.sayac += 1
    def bilgi(self):
        return f"Ben {self.ad}"
    @classmethod
    def toplam(cls):
        return cls.sayac

a = Ogrenci("Ada")
print(a.bilgi())
print(Ogrenci.toplam(), a.toplam())
try: Ogrenci.bilgi()
except TypeError as e: print("Hata:", e)
```

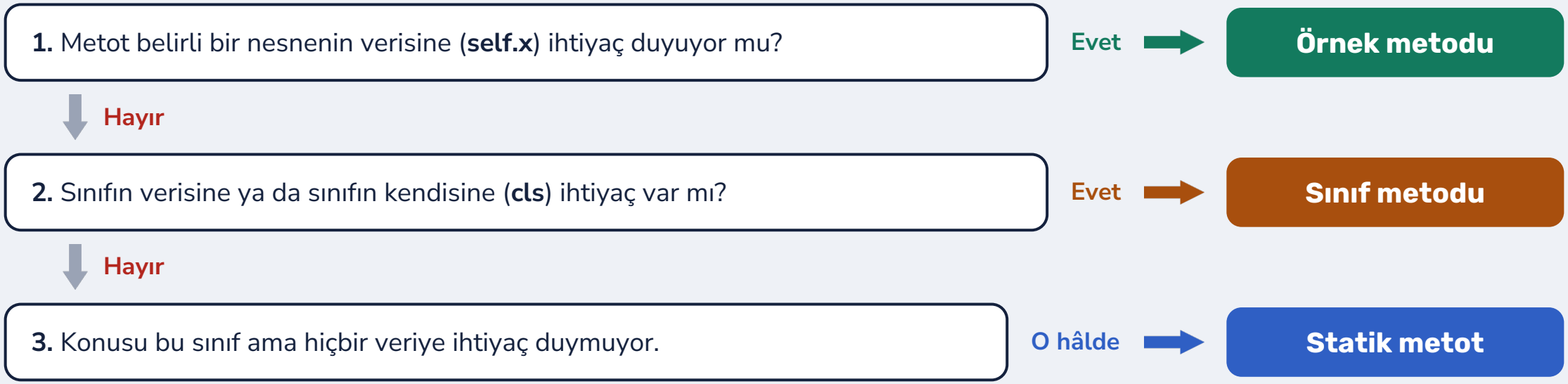
▷ ÇIKTI

```
Ben Ada
1 1
Hata: Ogrenci.bilgi() missing 1 required
positional argument: 'self'
```

Metot	nesne.m()	Sinif.m()
Örnek	Evet	Hayır, self yok
Sınıf	Evet	Evet
Statik	Evet	Evet

KARAR AĞACI

Hangi metot türünü seçmeliyim?



Konusu bu sınıf bile değilse? O zaman sınıfın dışında sıradan bir fonksiyon olmalı.

BAŞVURU KARTI

Üç metot türü tek tabloda

	Örnek metodu	Sınıf metodu	Statik metot
Dekoratör	yok	@classmethod	@staticmethod
İlk parametre	self (nesne)	cls (sınıf)	özel yok
Nesne verisi	Erişir	Erişemez	Erişemez
Sınıf verisi	Erişir	Erişir (cls.x)	Yalnızca adla
Tipik kullanım	Davranış, hesaplama	Sayaç, alternatif yapıcı	Doğrulama, yardımcı
Çağrı	a.bilgi()	Ogrenci.toplam()	Ogrenci.gecerli(5)

UYGULAMA · MİNİ PROJE

Ürün sınıfı: bugünün her parçası bir arada

urun.py

```python
class Urun:
    kdv_orani = 0.20
    adet = 0
    def __init__(self, ad, fiyat):
        if not Urun.gecerli_fiyat(fiyat):
            raise ValueError("Fiyat pozitif olmalı")
        self.ad = ad
        self.fiyat = fiyat
        Urun.adet += 1
    def kdvli_fiyat(self):
        return round(self.fiyat * (1 + Urun.kdv_orani), 2)
    @classmethod
    def kdv_degistir(cls, yeni_oran):
        cls.kdv_orani = yeni_oran
    @staticmethod
    def gecerli_fiyat(f):
        return f > 0
```

HANGİ PARÇA NE?

- **kdv_orani, adet**: sınıf değişkenleri (ortak ayar ve sayaç)
- **ad, fiyat**: örnek değişkenleri (her ürüne özel)
- **kdvli_fiyat()**: örnek metodu (self.fiyat gerekli)
- **kdv_degistir()**: sınıf metodu (herkesin oranı)
- **gecerli_fiyat()**: statik metot (veri gerekmez)

UYGULAMA · KULLANIM

Tek satırla tüm ürünlerin KDV'si değişti

urun.py (devam)

```
k = Urun("Kalem", 15)
d = Urun("Defter", 40)
print(k.kdvli_fiyat(), d.kdvli_fiyat())

Urun.kdv_degistir(0.10)
print(k.kdvli_fiyat(), d.kdvli_fiyat())

print("Ürün sayısı:", Urun.adet)
```

▷ ÇIKTI

```
18.0 48.0
16.5 44.0
Ürün sayısı: 2
```

Kalem

15 × 1.20 = 18.0
15 × 1.10 = 16.5

Defter

40 × 1.20 = 48.0
40 × 1.10 = 44.0

Neden çalıştı?

kdv_orani tek bir yerde duruyor. kdv_degistir() o tek değeri değiştirdi; her iki ürün de bir sonraki hesaplamada yeni oranı okudu.

1000 ürün olsaydı da tek satır yeterdi.

UYGULAMA · İZ SÜRME

Sınıf ve nesneler, adım adım

#	Satır	kdv_orani	adet	fiyatlar	Çıktı
1	sınıf tanımı	0.20	0	–	–
2	Urun("Kalem", 15)	0.20	1	k: 15	–
3	Urun("Defter", 40)	0.20	2	k: 15, d: 40	–
4	kdvli_fiyat() × 2	0.20	2	değişmedi	18.0 48.0
5	kdv_degistir(0.10)	0.10	2	değişmedi	–
6	kdvli_fiyat() × 2	0.10	2	değişmedi	16.5 44.0

Turuncu sütunlar sınıfta tek kopya; her satırda tüm ürünler için geçerli.

Yeşil sütun her nesnenin kendi değeri; KDV değişikliğinden etkilenmez.

SÖZLÜK

Bu haftanın terimleri

Terim	İngilizce	Anlamı
Örnek değişkeni	instance variable	self üzerinde duran, nesneye özel veri
Sınıf değişkeni	class variable	Sınıf gövdesinde tanımlanan, tüm nesnelerin paylaştığı veri
Gölgeleme	shadowing	Nesnede aynı adlı nitelik oluşup sınıftakini örtmesi
Örnek metodu	instance method	İlk parametresi self olan, nesneye bağlı metot
Sınıf metodu	class method	@classmethod ile işaretli, ilk parametresi cls
Statik metot	static method	@staticmethod ile işaretli, self/cls almayan yardımcı
Dekoratör	decorator	@ ile yazılan, fonksiyonun davranışını değiştiren etiket
Alternatif yapıcı	factory method	Farklı girdiden nesne üreten sınıf metodu

KONTROL LİSTESİ

Sınıfını yazdıktan sonra sor

- Nesneden nesneye değişen veri **self** üzerinde mi?
- Herkes için ortak veri **sınıf gövdesinde** mi?
- Sınıf değişkenini **Sinif.x** veya **cls.x** ile mi değiştiriyorum?
- Kişiye özel listeler **__init__** içinde mi oluşturuluyor?
- self'e ihtiyaç duyan her metot **örnek metodu** mu?
- Yalnızca sınıf verisi kullananlar **@classmethod** mu?
- Hiç veri kullanmayan yardımcılar **@staticmethod** mu?
- Komutlar durumu değiştiriyor, sorgular **değer döndürüyor** mu?

KENDİNİ SINA

Tahmin et: ekranda ne görünür?

Soru 1 • Gölge ve sınıf

```
class A:
    x = 1

a = A()
b = A()
a.x = 5
A.x = 10
print(a.x, b.x)
```

5 10

a kendi x'ini aldı (gölge). b hâlâ sınıfa bakıyor ve yeni değeri görüyor.

Soru 2 • Ortak sayaç

```
class Sayac:
    deger = 0
    def artir(self):
        Sayac.deger += 1

s1 = Sayac()
s2 = Sayac()
s1.artir(); s2.artir(); s2.artir()
print(s1.deger, Sayac.deger)
```

3 3

Üç artış da sınıftaki tek sayaca yazıldı; s1.deger de oradan okunuyor.

DİKKAT

En sık yapılan 6 hata

`self.sayac += 1`

Sınıf sayacı artmaz; nesnede yeni değişken oluşur. Doğrusu: **Sinif.sayac**

`a.okul = "X"`

Sınıfı değiştirmez, sadece a'da **gölge** oluşturur.

`notlar = [] (sınıfta)`

Tüm nesneler **aynı listeyi** paylaşır. __init__'e taşıyın.

`def toplam(cls):`

@classmethod unutuldu → **missing 1 required positional argument: 'cls'**

`self (statik içinde)`

Statik metot self almaz → **NameError: name 'self' is not defined**

`print(h.yatir(50))`

Komut metodu **None** döner; önce çağır, sonra niteliği yazdır.

ÖZET

Bugün öğrendiklerimiz, dört cümlede

Örnek değişkeni

self.x: nesne başına bir kopya; her nesnenin kendi verisi.

Sınıf değişkeni

Sınıf gövdesinde tek kopya; herkes okur, **Sinif.x** ile değiştirilir.

Arama ve gölge

Okurken önce nesne, sonra sınıf. Nesne üzerinden atama yeni bir gölge yaratır.

Üç metot türü

self → örnek, **cls** → sınıf metodu, hiçbiri → statik metot.

“Bu bilgi kime ait?” sorusu, verinin ve metodun yerini belirler.

PRATİK ÖDEV · GELECEK HAFTA

Doğru veriyi doğru yere koy

1 · Çalışan

asgari_ucret (sınıf), ad ve maas (örnek). **zam_yap(oran)**, **asgari_degistir()** ve **gecerli_maas()** yaz.

2 · Otomatik numara

Her Kitap nesnesi sıradaki id'yi alsın. **Kitap.toplam()** sınıf metodu kaç kitap olduğunu söylesin.

3 · Fabrika

Ogrenci.sozlukten({"ad": ..., "numara": ...}) alternatif yapıcısını yaz.

4 · Tuzak avı

Üç tuzağı (13–15. slaytlar) kendi kodunda üret, çıktıyı kaydet, sonra düzelt.

HAFTA 5

Kapsülleme: erişim belirleyiciler, getter/setter

Şu an herkes **u.fiyat = -100** yazabiliyor. Bunu nasıl engelleriz?

```
self._fiyat
self.__fiyat
@property
```